HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 02

GitHub Copilot Recap & Advanced Features

A fast recap of where Copilot came from and how its core works — then most of our time on the power-user layer: instructions, prompt files, custom agents, skills, MCP, hooks and agent-mode workflows.

DURATION
2.5 hours

FORMAT
Recap + 4 hands-on labs

PRIMARY TOOL
Copilot Enterprise in VS Code

How We'll Spend These Two and a Half Hours

A short recap, then a climb through Copilot's customisation and extension layers — with a lab at each step.

01

Background, Plans & Surfaces

15 min

From autocomplete to agents: Copilot's evolution, where it runs, and how plans and AI credits work.

02

Core Recap: Editor, Modes & Commands

20 min

Completions, next edit suggestions, inline chat, Ask/Plan/Agent, slash commands, @ and # context.

03

Instructions & Prompt Files

30 min

Always-on and file-scoped instructions, reusable prompt files — Lab 1: build a team instruction set.

04

Custom Agents & Agent Skills

40 min

Personas with tools and handoffs; packaged, on-demand know-how — Labs 2 and 3.

05

MCP, Hooks & Model Choice

15 min

Connecting external tools, deterministic guardrails, and picking a model per task.

06

Agent-Mode Workflows, CLI & Cloud Agent

30 min

Plan → implement → review end-to-end, beyond the IDE — Lab 4: compare against baseline.

Module 02 in the 12-Module Journey

The tool foundation every later module builds on.

FOUNDATION			BUILD & VALIDATE			EXTEND WITH TOOLS & AGENTS				GOVERN	CAPSTONE
1	2	3	4	5	6	7	8	9	10	11	12
Objectives & GenAI Recap	**Copilot Advanced**	Prompt & Context Eng.	Spec-Driven Dev (Spec Kit)	SDD to Full SDLC	Test Strategy & Validation	MCP & Custom MCP Server	PR Automation & LLM-Judge	No-Code Agents (Rovo)	Code Agents & Simple RAG	Observability, ROI & Gov.	Capstone & Roadmap

M03
Context engineering uses instructions, prompt files and #context

M04–05
Spec Kit runs as prompt files and agents in agent mode

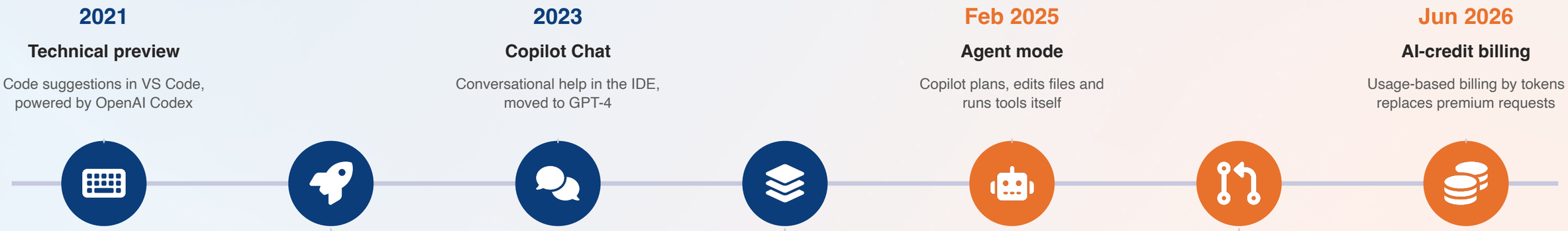
M07
Builds the custom MCP server you'll consume today

M10
Copilot SDK agents reuse the same agent concepts in code

Why this matters: everything you create today — instructions, prompt files, agents, skills, MCP config — is committed to the repo and reused in every module that follows.

From Autocomplete to Agents: Copilot's Evolution

Five years from ghost-text suggestions to autonomous agents that open pull requests.



2021
Technical preview
Code suggestions in VS Code, powered by OpenAI Codex

2022
General availability
Subscription service for individual developers

2023
Copilot Chat
Conversational help in the IDE, moved to GPT-4

2024
Multi-model choice
Pick among model families, e.g. Claude, Gemini

Feb 2025
Agent mode
Copilot plans, edits files and runs tools itself

May 2025
Coding agent
Async agent drafts PRs in the cloud — now the cloud agent

Jun 2026
AI-credit billing
Usage-based billing by tokens replaces premium requests

WHERE COPILOT RUNS TODAY

- VS Code ← today
- Visual Studio
- JetBrains IDEs
- Neovim
- GitHub.com
- Copilot CLI
- Cloud agent

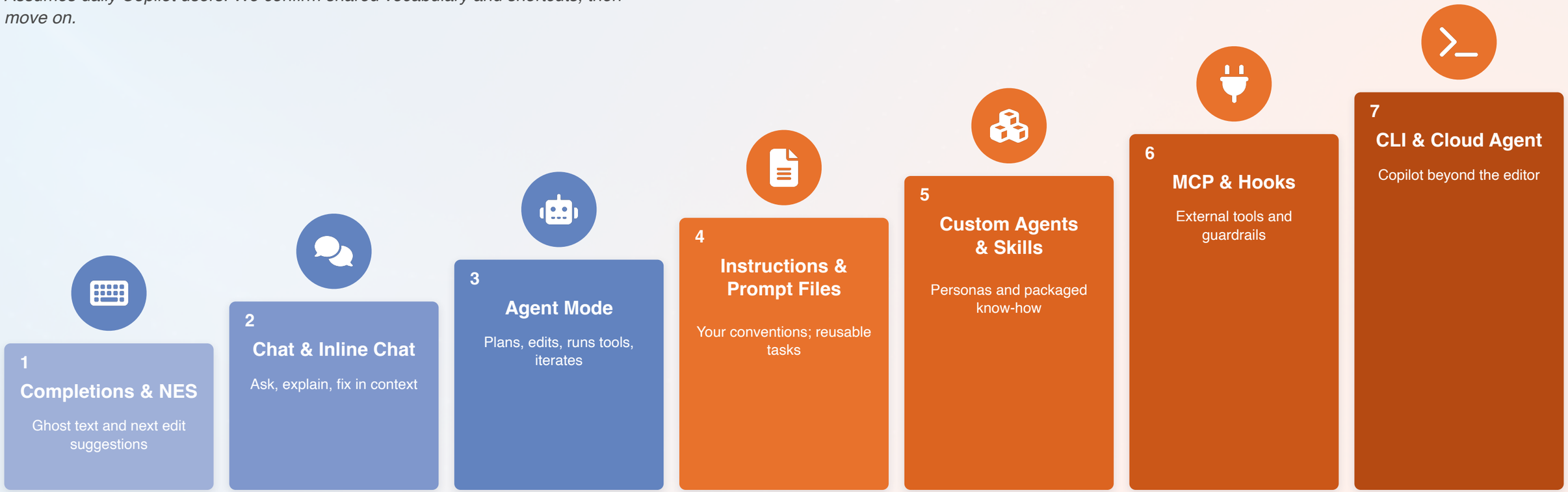
Today's Climb: Seven Rungs from Recap to Power User

Rungs 1–3 are a fast recap; rungs 4–7 are where the cohort spends most of its time.

RECAP — ~35 MIN

Assumes daily Copilot users. We confirm shared vocabulary and shortcuts, then move on.

ADVANCED — ~115 MIN, 4 LABS



Copilot Plans: What Changes from Free to Enterprise

Capabilities and controls by plan — billed through AI credits since June 2026. Prices intentionally omitted.

	INDIVIDUAL					ORGANISATION	
	Free	**Student**	**Pro**	**Pro+**	**Max**	**Business**	**Enterprise**
FOR	Individuals without org access	Verified students, free	Individual developers	Heavy individual users	Highest individual usage	Organisations	GitHub Enterprise Cloud
MODELS	Auto model selection only	Auto model selection only	A selection of models	Premium models	Priority premium models	Premium models	Priority premium models
	Limited agents; 2,000 completions / month	No third-party agents	Monthly AI-credit allowance	Larger credit allowance	Highest individual allowance	Central management & policy; pooled credits	All of Business + enterprise-grade features; largest pooled allowance

Programme platform

AI credits

Chat, agents and premium models consume credits by tokens (input, output, cached) at each model's rate.

Always included

Code completions and next edit suggestions don't consume credits on any plan.

Admin budgets

Org plans pool credits; admins set budgets per enterprise, cost centre or user.

Core Recap: Four Ways to Work, Four Levels of Autonomy

In the editor, Copilot suggests, converses or acts — you choose how much control to hand over.

IN THE EDITOR

Code completions

Tab / Esc

Ghost-text suggestions as you type

Next edit suggestions

Tab

Predicts your next edit and where it goes

Inline chat

Ctrl+I

Prompt in place: editor, terminal, notebook

Chat view

Ctrl+Alt+I

Multi-turn chat with agents, tools, context

IN CHAT — PICK A MODE (Ctrl+Shift+I)

Ask	Answers and explains using your code as context. Doesn't change files.
Plan	Researches the codebase and writes an implementation plan, then hands off.
Edit	Proposes edits across files you name; you review and keep or discard.
Agent	Takes a goal: finds files, edits, runs terminal and tests, iterates until done.

↓ *more autonomy, more need to verify*

The Command Vocabulary: /, @ and

Slash runs a task, @ picks a domain expert, # attaches precise context.

Slash commands

Run a task or a reusable prompt

`/explain`	explain selected code
`/fix`	propose a fix
`/tests`	generate tests
`/doc`	add documentation
`/init`	draft project instructions
`/<name>`	run a prompt file or skill

Chat participants

Route the question to a domain

`@workspace`	the current workspace
`@github`	repos, issues, pull requests
`@terminal`	shell and terminal
`@vscode`	editor settings and features

Context references

Attach exactly what matters

`#codebase`	search the whole codebase
`#file / #symbol`	a specific file or symbol
`#selection`	current selection
`#changes`	uncommitted changes
`#problems`	errors and warnings
`#fetch`	content of a web page

KEYBOARD SHORTCUTS (Windows/Linux — use ⌘ on macOS)

`Ctrl+Alt+I`	Open chat	`Ctrl+I`	Inline chat	`Ctrl+Shift+I`	Switch agent	`Ctrl+Alt+.`	Model picker	`Ctrl+N`	New chat	`F2`	AI rename

The Customisation Stack: Seven Layers You Commit to the Repo

Each layer answers a different question — and is triggered a different way.

		TRIGGERED BY
Always-on instructions	*What should Copilot always know?*	**Every chat request**
File-scoped instructions	*What applies to these files?*	**Matching applyTo glob**
Prompt files	*How do we do this task?*	**You type /name**
Custom agents	*Who should do it, with what tools?*	**You pick the persona**
Agent skills	*What specialist know-how is needed?*	**Model loads when relevant**
MCP servers	*Which external systems can it reach?*	**Agent calls a tool**
Hooks	*What must always (never) happen?*	**Lifecycle event fires**

repo layout

```
.github/
├ copilot-instructions.md
├ instructions/
│  └ api.instructions.md
├ prompts/
│  └ add-tests.prompt.md
├ agents/
│  └ reviewer.agent.md
├ skills/
│  └ dotnet-tests/SKILL.md
└ hooks/
   └ audit.json
.vscode/mcp.json
AGENTS.md  # cross-agent option
```

Custom Instructions: Teach Copilot Your Team's Rules Once

Always-on and file-scoped Markdown that's added to chat and agent requests — so you stop repeating yourself.

.github/copilot-instructions.md

```
# Project: Zone Control API
- .NET 8 Web API + React front end
- Build: dotnet build  Test: dotnet test
- Validation uses FluentValidation in /Validation
- Never change public API contracts without a spec
  (why: field devices depend on them)
```

.github/instructions/api.instructions.md

```
---
applyTo: "src/Api/**/*.cs"
description: "Controller and validator rules"
---
- Return ProblemDetails for 4xx/5xx
- Log with ILogger<T>; no Console.WriteLine
```

Always-on

copilot-instructions.md or AGENTS.md — included with every chat and agent request in the repo.

File-scoped

*.instructions.md with an applyTo glob, or a description so Copilot loads it when the task matches.

Three scopes

User (personal) · Workspace (committed, shared) · Organisation (managed in GitHub). Sources add together.

Generate a draft

Type /init for project-wide instructions or /create-instructions for a scoped file — then review it.

Write them well: short, single-purpose rules; say why; show a preferred and an avoided pattern; skip what linters already enforce. *Note: instructions don't affect inline completions.*

Prompt Files: Turn a Good Prompt into a Team Command

A .prompt.md file is a reusable, parameterised task you run with /name — on demand, not always on.

.github/prompts/add-endpoint-tests.prompt.md

```
---
description: 'Write xUnit tests for an API endpoint'
agent: 'agent'
tools: ['search/codebase', 'runTests']
argument-hint: 'scenario to focus on'
---
Write xUnit tests for ${selection}.
Cover valid, boundary and invalid input, focusing on
${input:scenario:out-of-range setpoint}.
Follow [our test conventions](../instructions/tests.instructions.md).
Run the tests with #tool:runTests and fix failures.
```

/add-endpoint-tests scenario=null zone id

ANATOMY

Frontmatter	description, agent (ask / agent / custom), model, tools, argument-hint — all optional
Variables	${selection}, ${file}, ${workspaceFolder}, ${input:name:placeholder}
References	Link instruction files with Markdown links; call tools with #tool:<name>
Run it	/name in chat, Chat: Run Prompt, or the play button in the editor
Scope	.github/prompts/ for the team; your profile for personal prompts

Instructions = how we always work. Prompt files = how we do this one task.

Lab 1: Build a Team Instruction Set and a Prompt File

LAB 1 · 15 min

Make Copilot follow your conventions without restating them in every prompt.

Goal Commit always-on and file-scoped instructions plus one reusable prompt file to the sample repo.

STEPS

1. Ask chat a baseline question (“Add validation to UpdateSetpoint”) and save the answer for comparison.
2. Run /init to draft .github/copilot-instructions.md. Trim to non-obvious rules: stack, build/test commands, conventions, reasons.
3. Add api.instructions.md with applyTo: "src/Api/**/*.cs" — validation, error-handling and logging patterns.
4. Create add-endpoint-tests.prompt.md using ${selection} and ${input:scenario}; run it with /add-endpoint-tests.

FILES YOU CREATE

```
repo/
.github/
  copilot-instructions.md
.github/instructions/
  api.instructions.md
.github/prompts/
  add-endpoint-tests.prompt.md
```

DONE WHEN

Re-asking the baseline question gives an answer that follows your conventions without you naming them, and the References list in chat shows your instruction files were used. /add-endpoint-tests generates tests that pass.

Custom Agents: Personas with Their Own Tools, Model and Handoffs

An .agent.md file defines who does the work, what it may touch, and where it hands off next.

.github/agents/reviewer.agent.md

```
---
name: Reviewer
description: Review changes for bugs and security
tools: ['search', 'usages']    # read-only
handoffs:
  - label: Apply Fixes
    agent: agent
    prompt: Apply the fixes suggested above.
---
Review the change against our instructions.
Do not edit files. List each finding with its
location, severity and a suggested fix.
```

Also: model (one or a prioritised list), agents (allowed subagents), user-invocable, disable-model-invocation, target, hooks.

HANDOFFS CHAIN AGENTS INTO A WORKFLOW

Plan

Read-only research → implementation plan

handoff: Start Implementation

Implement

Agent edits, runs build and tests

handoff: Review Changes

Review

Reviewer finds issues, no edit tools

Use them: pick from the agents dropdown, type /agents to manage, or /create-agent to generate one. Formerly “custom chat modes”.

Lab 2: Configure a Read-Only Reviewer Agent

LAB 2 · 15 min

A persona whose tools make the wrong behaviour impossible, with a handoff back to implementation.

Goal Create a custom Reviewer agent with read-only tools and an “Apply Fixes” handoff, and use it on a flawed change.

STEPS

1. Run Chat: New Custom Agent (or /create-agent) and save it as .github/agents/reviewer.agent.md.
2. Set tools to read-only (search, usages). In the body, point to your instruction files and define the finding format.
3. Add a handoff: label “Apply Fixes” → agent: agent, with a pre-filled prompt.
4. Select Reviewer, review the deliberately flawed branch, then click Apply Fixes and let Agent mode fix it.

FILES YOU CREATE

```
repo/
.github/agents/
  reviewer.agent.md
```

DONE WHEN

Reviewer lists findings with location, severity and fix, and edits no files. The Apply Fixes button switches to Agent mode with the prompt pre-filled, and the fixed code passes the tests.

Agent Skills: Packaged Know-How That Loads Only When Needed

A skill is a folder: SKILL.md instructions plus scripts, templates and examples — an open standard across agents.

.github/skills/dotnet-tests/

```
SKILL.md          # name, description, steps
scripts/run-filtered.ps1
templates/test-class.cs
examples/zone-service-tests.cs
```

SKILL.md

```
---
name: dotnet-tests
description: Run, write or fix .NET unit
  tests. Use when asked about xUnit tests.
---
1. Run scripts/run-filtered.ps1 for the class
2. New tests start from templates/test-class.cs
```

PROGRESSIVE LOADING — WHY SKILLS SCALE

1 Always: name + description

Only a line or two per skill sits in context, so you can have many skills.

2 When relevant: SKILL.md body

Loaded when the task matches the description, or when you type /dotnet-tests.

3 When referenced: bundled files

Scripts, templates and examples are opened only when the steps call for them.

ONE SKILL, MANY AGENTS

- VS Code
- Copilot CLI
- Cloud agent
- Other agents

Project skills: .github/skills/ · Personal: ~/.copilot/skills/ · Standard: agentskills.io

Lab 3: Package a Reusable Skill

LAB 3 · 15 min

Capture a repeatable workflow once so every engineer — and every agent — can use it.

Goal Create a dotnet-tests skill with a bundled script and template, and confirm Copilot loads it on its own.

STEPS

1. Create .github/skills/dotnet-tests/ with a SKILL.md: name, and a description saying what it does and when to use it.
2. Write the steps in the body. Add scripts/run-filtered.ps1 (dotnet test with a --filter) and templates/test-class.cs.
3. Without naming the skill, ask Agent mode: “Add and run unit tests for ZoneService.” Check it was loaded.
4. Invoke it directly with /dotnet-tests for AlarmService. Tighten the description if it didn't trigger in step 3.

FILES YOU CREATE

```
repo/

.github/skills/dotnet-tests/
  SKILL.md
  scripts/run-filtered.ps1
  templates/test-class.cs
```

DONE WHEN

In step 3 the skill loads without being named, the new tests follow the template, and the script runs them. /dotnet-tests works as a slash command.

Which Customisation, When?

Same goal — better output with less prompting — four different tools.

	Custom instructions	Prompt files	Custom agents	Agent skills
Answers	How we always work	How we do this task	Who does it, with what tools	Specialist capability + resources
Triggered by	Every request / applyTo glob	You type /name	You select it (or a handoff)	Model, when relevant — or /name
Contains	Markdown rules	Markdown + variables + tools	Persona, tools, model, handoffs	SKILL.md + scripts, templates
Context cost	Always in context	Only when run	While selected	Tiny until loaded
Best for	Stack, standards, guardrails	Repeated tasks (tests, docs)	Roles: planner, reviewer	Workflows: test runs, migrations
Example	`copilot-instructions.md`	`add-endpoint-tests.prompt.md`	`reviewer.agent.md`	`skills/dotnet-tests/`

Rule of thumb: rules go in instructions; tasks in prompt files; roles in agents; workflows with resources in skills. If you paste the same thing into chat twice, it belongs in one of these.

MCP: Give Copilot Governed Access to Your Systems

The Model Context Protocol connects agents to external tools and data through a standard interface.

You

"List open bugs for the setpoint API and draft a fix"

Copilot agent

MCP client in VS Code: discovers tools, asks approval, calls them

- **GitHub MCP server** issues, PRs, repos
- **Database server** read-only schema & queries
- **Docs / wiki server** design notes, runbooks
- **Your custom server** built in Module 07

Tools	actions the agent can call
Resources	data it can read
Prompts	templates the server offers

.vscode/mcp.json

```
{ "servers": {
    "github": { "type": "http",
      "url": "https://api.githubcopilot.com/mcp" }
} }   // stdio servers use command + args
```

Trust & approval

Trust each server on first start; approve tool calls per call, session or workspace.

Enterprise policy

chat.mcp.access and Copilot policy control which servers may run at all.

Keep it lean

Up to 128 tools per request — enable only the servers a task needs.

Hooks and Model Choice: Control What Happens and What It Costs

Hooks add deterministic guardrails to agent sessions; model choice trades quality, speed and credits.

HOOKS (PREVIEW) — SHELL COMMANDS ON LIFECYCLE EVENTS



.github/hooks/audit.json

```
{ "hooks": {
  "PostToolUse": [{
    "type": "command",
    "command": "./log-tool.sh"
  }] } }
```

- ✓ **Block destructive commands**
- ✓ **Format or lint after edits**
- ✓ **Audit-log every tool call**
- ✓ **Inject project context**

Unlike instructions, hooks always run — they don't depend on the model choosing to comply.

MODEL CHOICE (Ctrl+Alt+.)

Auto

Let Copilot pick. A sensible default for everyday chat.

Fast, lighter models

Explanations, small edits, boilerplate. Lower latency, fewer credits.

Stronger reasoning models

Planning, multi-file refactors, tricky debugging. Slower, cost more per token.

Pin it: prompt files and custom agents can set model — agents accept a prioritised list. Under AI credits, the model you pick changes the cost of every run.

Putting It Together: An Agent-Mode Workflow End to End

Every layer from today working on one task: plan, implement, review, ship.

1 Plan
Plan agent

Researches with #codebase, writes a step plan. No edits.

2 Implement
Agent mode

Edits files, runs build and tests in the terminal, iterates on failures.

3 Review
Reviewer agent

Checks against instructions; read-only; hands off fixes.

4 Ship
You

Review the diff, commit, open the PR — or /delegate to the cloud agent.

WHAT FEEDS EVERY STEP

- **Instructions**
- **Prompt files**
- **Skills**
- **MCP tools**

GUARDRAILS

- ✓ Approve tool and terminal calls
- ✓ Checkpoints to roll back edits
- ✓ Hooks enforce hard rules
- ✓ Human reviews every diff

Beyond the Editor: Copilot CLI and the Cloud Agent

The same instructions, agents, skills and MCP servers — in your terminal, or working on its own in the cloud.

COPILOT CLI — AGENTIC HELP IN THE TERMINAL

```
terminal
$ copilot
> /model              choose the model
> /agent reviewer     select a custom agent
> /mcp                manage MCP servers
> /add-dir ./src       grant directory access
> /usage              session usage metrics
> /delegate add setpoint validation
  → hands off to the cloud agent as a PR
```

Good for: scripted or terminal-first work, quick repo questions, and running agents outside the IDE.

CLOUD AGENT — ASYNCHRONOUS, PR-BASED

Assign a task	From GitHub (e.g. an issue), VS Code or /delegate
Works in the cloud	Own environment: branch, edits, build and test
Opens a draft PR	With a log of what it did
You review & iterate	Comments trigger new commits; you merge

Never pushes to protected branches — your branch rules and review gates still apply.

Portable by design: skills, custom agents and MCP config you commit today travel with the repo into the CLI and the cloud agent. That's the payoff of putting know-how in files, not in people's heads.

Lab 4: Agent-Mode End to End vs. a Chat-Only Baseline

LAB 4 · 25 min

Run the same feature two ways and measure the difference — evidence for your adoption case.

Goal Implement one small feature first with chat and inline only, then with the full stack: Plan → Agent → Reviewer.

STEPS

1. Baseline: implement “reject out-of-range setpoints with ProblemDetails + tests” using chat and inline only. Record time and iterations.
2. Reset the branch. Start in the Plan agent, review the plan, then hand off to Agent mode to implement and run tests.
3. Switch to your Reviewer agent; use Apply Fixes for any findings. Let the dotnet-tests skill handle test runs.
4. Fill in the comparison: time, iterations, conventions followed, tests passing, and credits used. Share one insight.

FILES YOU CREATE

```
repo/

Uses Labs 1-3:
  copilot-instructions.md
  api.instructions.md
  reviewer.agent.md
  skills/dotnet-tests/
comparison-worksheet.md
```

DONE WHEN

Both runs produce a passing build, and the worksheet has a row for each approach covering time, iterations, rule violations and credits. Each pair names one place the advanced flow helped and one place it cost more.

Module 02 Outcomes & What's Next

What should be true, and committed to the repo, as we leave.

- ✓ Copilot's evolution, surfaces, plans and AI-credit billing are understood
- ✓ Completions, chat modes and /, @, # commands are used fluently
- ✓ A team instruction set and a prompt file are committed
- ✓ A read-only reviewer agent with a handoff is configured and used
- ✓ A reusable skill loads on demand; MCP and hooks are understood as extension and guardrail layers
- ✓ An agent-mode task has been run end to end and compared against a baseline

NEXT: MODULE 03 — PROMPT ENGINEERING RECAP & CONTEXT ENGINEERING · 2 HOURS

From prompt-only to structured context: selection, layering, compression and token-aware practice — compared on quality, iterations and tokens.

THE CLIMB TODAY

Completions → Chat → Agent mode → Instructions → Agents & skills → MCP & hooks → CLI & cloud

Questions & Discussion

Module 02 — GitHub Copilot Recap & Advanced Features